

申請人

葉 陽 甫

就讀學校　國立花蓮高級中學

申請校系　**國立陽明交通大學 電機工程學系（116 學年度特殊選才）**

自 傳

從實體世界開始



▲ 智慧遮雨棚實體

A 國中｜多功能智慧遮雨棚

這是我第一次把程式寫進真實裝置。我負責 Arduino 接線、程式與 App Inventor 手機控制：感測器讀到雨滴、溫溼度與距離，Arduino 依三種情境判斷，馬達再收放遮雨棚。任何一處接線或機構出錯，整套系統就沒有反應——程式必須和真實環境一起工作。

成果｜2023 IEYI 臺灣選拔賽　國中組便利生活類銀獎

▶ 延伸資料｜多功能智慧遮雨棚發明展資料 ↗

B 高中｜防呆警報器

我原本一次把整套系統接起來，紅外線訊號卻一直讀不到，也不知道問題在哪一層。後來改成每個模組單獨驗證，再逐一加入人體感測器、蜂鳴器與伺服馬達，最後才整合，從此養成「先拆開、逐層驗證、再整合」的習慣。

▶ 延伸資料｜智慧節能防呆警報器 ↗

▶ 延伸資料｜操作影片 ↗



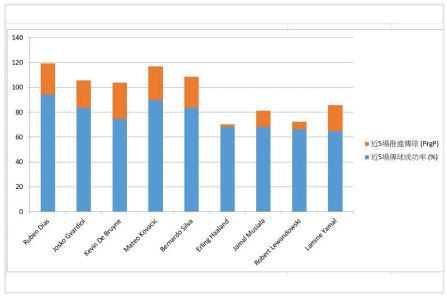
▲ 紅外線、PIR、蜂鳴器與伺服馬達

C 高一｜FBref 球員數據分析

因為想比較足球員表現，我自學 Python、pandas 與爬蟲整理 FBref（足球數據網站）資料。單一球員成功後我直接擴大請求，IP 卻被限制；VPN 只撐一天，最後才理解 Rate Limiting，改為加入請求間隔。

▶ 延伸資料｜專題報告書 ↗

▶ 延伸資料｜FBref 分析程式碼與資料 GitHub ↗



▲ 球員近五場數據比較

裝置 → 程式 → 外部系統

FBref 讓我發現，即使程式沒有 bug，不了解所依賴的外部系統仍會失敗。問題從「程式怎麼寫」，變成「整個系統怎麼運作」。

從教學問題，長成 AI 系統

延伸資料 ｜ ▶ 專案 GitHub ↗ ▶ 作品 Demo 頁面 ↗

高二時我想要參加育秀盃，指導老師要我們先找身邊真正存在的問題。我問在高職教數學的父親，他說同一班學生程度差距很大，老師難以在同一堂課替不同學生安排不同進度的練習與回饋。這成為我與兩位同學製作「自適應性輔助學習系統」的起點。在三人團隊中，我主要負責教材進入系統到學生作答回傳的資料主線，以及 Code-as-Content 的程式題生成；後續也由這條工作延伸出 AST Healer 的研究。

A 學生答完之後，下一題往哪走？

作答 → 能力診斷 → **適性決策** →

- 掌握 → 繼續下一題
- 不足 → 補前置知識 → AI 助教 → 再作答

為了回答這個問題，系統需要知識結構、足夠的題目、作答紀錄與引導，於是逐步長出下列模組。

知識結構
年級 → 章節 → 知識節點

教材匯入
PDF／Word → 解析 → 教師審核

Code-as-Content
模型依題目生成 Python → 動態生成題目

學生作答
紀錄 → AI 引導 → 後續學習



▲ 教師端：班級管理與教學工具



▲ 學生端：依知識節點學習

▶ 延伸資料｜自適應輔助學習系統介紹 ↗

B 同一套系統，持續演進

扶輪盃時，系統第一次成為可完整操作的原型；之後我們持續整合教材、題庫、作答與適性流程，到育秀盃已是一套較完整的學習系統。

2026 扶輪盃高中 AI 競賽 — 早期原型
全國高中組第一名

→

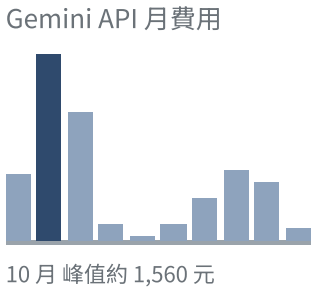
第 23 屆育秀盃 — 系統完整成果
全國高中職 AI 應用類金獎

▶ 延伸資料｜育秀盃官方作品介紹（含影片） ↗

系統維運問題，成為研究起點

A 研究起點｜大型模型的成本與延遲

學習系統的 Code-as-Content 每次生成程式，都要呼叫雲端大型模型；token、API 成本與等待時間隨使用持續累積。我想把部分工作移給本地小模型，實測後卻發現：小模型的程式生成成功率並不穩定。



B 研究問題

怎麼讓小模型少做一點，卻做得更可靠？

我觀察到，同一數學單元的程式結構與常見錯誤會反覆出現，於是把問題拆成兩個可以驗證的子問題：

Q1｜生成前
哪些重複工作可以事先抽離？

→ **Scaffolding／Domain Functions**

先整理好題型規則、工具、參數來源與部分流程，降低模型自己規劃的負擔。

Q2｜生成後
哪些局部錯誤可用固定規則修復？

→ **AST Active Healer**

只修固定規格能唯一判斷修法的局部錯誤，把能確定修法的錯誤交給規則處理。

Healer 不是自動修好所有錯誤；無法確定修法時就不介入。

C 初版驗證

初版以三種模型，分為基礎提示組、鷹架提示組、鷹架提示 + Healer 組做消融實驗，並以自建評估指標比較。以下為三個不同模型在自建指標的平均分數：

Bare	→	+ Scaffolding	→	+ Full Healer
41.00		**94.39**		**97.41**

MCRI（Math Code Reliability Index）為自建評估指標；97.41 是 MCRI 分數，不是通過率。

三種模型都出現相同的改善趨勢。

▶ 延伸資料｜旺宏科學獎完整成果報告 ↗　▶ 延伸資料｜東區科展作品說明書 ↗

當時，我以為這些結果已經足以支持方法有效。

97.41 是怎麼得到的？ 收到評審提問後，我重新驗證

校內科展 **優勝** → 第 66 屆東區科展 **優等** 未晉級全國 → 第 25 屆旺宏科學獎 **全國決賽 20 強** 全領域 846 件 → 入圍之後 **收到評審建議**

A 初版驗證的三個缺口，以及下一版怎麼改

① 測試資料獨立嗎？	→ 開發／測試分離，評估前凍結規則
② Healer 修好了什麼？	→ 逐筆追蹤每個原始失敗
③ 自建 MCRI 夠當證據嗎？	→ 聚焦 Healer，改用 PASS／FAIL

B 重新驗證：Math16 16 道會考題 × 3 模型 × 4 Prompt × 5 seeds = 960 單元

4 題｜規則開發 240 單元 → **凍結 Prompt 與 Healer 規則** → **12 題｜保留題正式測試** 720 單元 • Healer 不讀答案 • 同題不跨組

720 正式測試 → **342** 原始失敗 → **42** Healer 實際修改 → **9** 失敗 → 通過 = **2.63%** 安全救援率

2.63% 與初版 MCRI 97.41 是不同指標，不能直接比較。

為什麼只有 2.63%？342 個原始失敗中，144 個（約 42%）是答案或演算法錯誤，需要重新理解題意或重新解題，本來就不在 Healer 的介入範圍。

能修｜輸出格式 • 局部結構 • 資料流 • 固定 API 規格可判定的錯誤

不修｜數學答案 • 核心演算法 • 題意理解 • 需重新推理的錯誤

C 新的發現｜主要差異來自模型能力與流程設計

補齊 Domain Functions 後，只給工具選單仍不夠；加上參數來源、呼叫順序、資料流與答案組裝的完整流程，部分中小型模型才進一步改善。

Domain Functions
提供領域工具

完整流程 Scaffolding
降低規劃負擔

Active Healer
只修可安全判定的錯誤

研究做到這裡，我開始看到 Edge AI 的潛力

我原本是為了降低 API 成本與延遲才嘗試本地小模型；研究後看到，不同能力的模型可以透過工具與流程分工承接不同工作。如果部分工作能由本地或更靠近裝置的一端承接，系統可以有不同的設計方式，這讓我開始注意 Edge AI 背後的執行平台、微處理機與嵌入式系統。

我的研究已開始探索雲端大型模型與本地小模型的分工，但還沒有進一步做到裝置端 Edge AI 部署，也沒有做過 FPGA 或 AI 晶片。

延伸能力｜英文溝通 × 足球競技

A 英文溝通　英文是我查技術資料、讀論文的重要工具

TOEIC 多益
945／990
金色證書・聽力 485／閱讀 460

GEPT 全民英檢
中高級 四項通過
聽 111・讀 87・說 80・寫 80

校內英文成績
平均 91｜前 4%
高一至高二英文平均

做 AI 與程式研究後，查模型文件、API 說明和國外技術文章時，我多半直接看英文原文；指導老師要我讀研究論文，我也能直接讀英文版本。演說和作文的練習，則讓我習慣先把內容整理清楚，再說出來、寫出來。

- 花蓮縣英語文閱讀能力競賽｜英文作文高中 A 組甲等
- 校內英語演說｜初賽一年級第一名、複賽全年級第四名
- 校內英文作文競賽｜二年級第二名





▲ TOEIC 成績單／GEPT 中高級合格證書

B 足球競技　從國小一路踢到高中的長期投入

113 學年度 中等學校足球聯賽（5 人制）
全國第六名
全國 63 隊 → 複賽 16 強 → 決賽 8 強

→

114 學年度 中等學校足球聯賽（5 人制）
全國第五名
全國 57 隊 → 複賽 16 強 → 決賽 8 強

- 兩年皆為正式主力球員（背號 42），連續兩年打入全國決賽 8 強
- 國三擔任花崗國中男子足球隊隊長，出賽全國國男甲級聯賽
- 全國青年盃足球錦標賽（11 人制）公開組第五名



▲ 114 學年度全國決賽，花蓮高中代表隊合影

高中做專題、準備科展和旺宏的同時，我一直維持校隊訓練與全國聯賽。長期踢足球也影響了我學程式的方向：高一時，我就是因為想比較球員表現，才開始用 Python 整理 FBref 的足球數據。

▶ 延伸資料｜114 學年直播精彩片段 ↗

申請動機：從研究看到 Edge AI 的下一步

01 研究把我帶到哪裡

AI 學習系統 → 大型模型成本／延遲 → 本地小模型 → Scaffolding／Healer → 不同能力模型如何分工 → **Edge AI**

我的研究原本是軟體與模型的問題。做到本地小模型之後，我開始看到：如果部分智慧能力能靠近裝置端執行，問題就會從模型本身，延伸到它所依賴的運算平台。

02 為什麼因此往電機深入

我現在有的

Python・AI 系統・模型串接
系統整合・研究驗證

理解 Edge AI 還缺的

電路・數位邏輯・微處理機
計算機結構・嵌入式系統

目前的研究已經開始處理雲端大型模型與本地小模型如何分工；做到這一步後，我開始想知道，如果模型真的往裝置端移動，還需要理解哪些底層問題。

03 為什麼是陽明交大電機

我在 Arduino 專題裡接過感測器、寫過控制程式，但當時只知道怎麼把模組接起來，還無法解釋電路與數位系統背後的原理。後來做到本地小模型，我又開始碰到另一層問題：模型最後仍要在處理器、記憶體與實際運算平台上執行。

陽明交大電機的領域涵蓋人工智慧與計算機工程、AI 機器人、資訊通訊等方向，讓我可以先把電路、電子與數位系統的基礎補起來，再逐步理解 AI 與程式落到實際運算平台時發生了什麼。

微算機原理與實驗、嵌入式系統技術實驗等實作課程，正好對應我目前還沒有系統學過、卻最想往下理解的一層。我希望先真正理解處理器、數位系統與嵌入式平台，再決定自己是否繼續往 Edge AI 深入。

讀書計畫 我現在的位置，以及下一步怎麼補

A 目前會做的

實體控制	感測器・接線・伺服馬達・分段除錯
Python／資料	爬蟲・pandas・xlwings・Rate Limiting
大型 AI 系統	教材匯入・Code-as-Content・Web・模型串接
研究／可靠度	Scaffolding・AST Healer・獨立保留測試

B 還沒學完整的，以及我怎麼發現的

資訊 **資料結構／演算法**	看得懂程式，也能把系統拆開；但把問題轉成演算法並穩定寫出來，還不夠熟。 APCS 觀念 92 分／5 級　實作 135 分／2 級
計算系統 **計算機結構／作業系統**	做本地小模型後，我開始想理解記憶體、處理器與運算資源實際如何被使用。
電機 **電路／數位邏輯／微處理機／嵌入式**	Arduino 時碰過感測器、接線與伺服馬達，但還沒有系統性學過電路、數位邏輯與微處理機。

C 三階段學習路徑

第一階段 **建立底層基礎**	先建立電路、電子、數位系統，以及程式與資料結構的基礎，把 Arduino 時期靠實作摸索出來的經驗，重新用正式的電機與計算基礎理解一次。
第二階段 **理解運算平台**	進一步學習微處理機、計算機結構與嵌入式系統，並透過微算機原理與實驗、嵌入式系統技術實驗等課程，理解程式與模型實際如何使用處理器、記憶體與運算資源。
第三階段 **用實作專題確認方向**	把高中研究留下的問題帶進專題，實際比較雲端、本地與裝置端的任務分工，再確認自己是否要繼續往 Edge AI、人工智慧與計算機工程或嵌入式系統深入。

D 把系統繼續做下去

目前系統已有學生實際使用；接下來希望把版本整理得更穩定並正式上線，持續用真實使用紀錄與外部競賽檢驗。

InnoServe
資訊系統／教育 AI 應用

經濟部智慧創新大賞
Best AI Awards｜AI 技術與實際應用

聯發科技「智在家鄉」
花蓮在地教育應用